

Matrix Exponentiation

In competitive programming, we often face problems where:

- There is a linear recurrence
- Or a DP transition that depends on a fixed number of previous states
- And n (the step, index, or time) is very large (up to $1e9$, $1e18$, etc.)

Example patterns:

- Fibonacci numbers
- Tiling DP
- Counting paths
- State transitions in automata
- DP with constant state size but huge steps

Matrix exponentiation is a tool to compute such recurrences in $O(k^3 \log n)$ instead of $O(n)$, where k is the number of states.

What is a Linear Recurrence?

Let's start with the most fundamental object.

A recurrence is called linear if:

$$f(n) = c_1 f(n-1) + c_2 f(n-2) + \dots + c_k f(n-k)$$

Where:

- Coefficients c_i are constants
- No multiplication between $f(\dots)$ terms
- No conditionals

Example 1: Fibonacci

$$F(n) = F(n-1) + F(n-2)$$

Example 2: Tribonacci

$$T(n) = T(n-1) + T(n-2) + T(n-3)$$

Example 3: DP-like recurrence

$$dp(n) = 2dp(n-1) + 3dp(n-3)$$

This structure is the key requirement for matrix exponentiation.

Let's write Fibonacci in the most boring DP way:

$$F(0) = 0, F(1) = 1$$

$$F(n) = F(n-1) + F(n-2)$$

This is a linear recurrence of order 2, meaning:

- $F(n)$ depends on 2 previous values
- The coefficients (1, 1) are constants

From a DP mindset:

$$F[i] = F[i-1] + F[i-2];$$

This is easy for small n , impossible for $n \approx 10^{18}$.

State $\neq$ Single Value

Most beginners think:

"My state is $F(n)$ "

That's wrong for recurrence thinking.

Correct thinking:

"My state is the minimum set of values needed to go to the next step."

So we define the state vector:

$$S(n) = \begin{bmatrix} F(n) \\ F(n-1) \end{bmatrix}$$

This is not arbitrary.
This is the smallest self-sufficient state.

Express Next State Using Current State

Now we ask the most fundamental question:

"Can I compute $S(n+1)$ using $S(n)$ only?"

Let's try.

First component:

$$F(n+1) = F(n) + F(n-1)$$

Second component:

$$F(n) = F(n)$$

So:

$$S(n+1) = \begin{bmatrix} F(n) + F(n-1) \\ F(n) \end{bmatrix}$$

Turn This Into a Linear Transformation

Now comes the matrix idea, but we won't assume it — we'll discover it.

We want:

$$S(n+1) = M \cdot S(n)$$

So:

$$\begin{bmatrix} F(n+1) \\ F(n) \end{bmatrix} = \begin{bmatrix} ? & ? \\ ? & ? \end{bmatrix} \begin{bmatrix} F(n) \\ F(n-1) \end{bmatrix}$$

Transition Matrix

Forget matrices for a moment.

Think like this:

"I have a small DP state.
I know how to compute the next state from the current state.
I want to jump many steps."

A transition matrix is just a compact way to store all the coefficients needed to compute the next state.

The Universal Method (Memorize This Thought Process)

Whenever you see a recurrence or DP:

Step 1 — Decide the state

Ask:

"What minimum values do I need to compute the next step?"

Step 2 — Write next_state in terms of current_state
Do this normally, without matrices.

Step 3 — Convert coefficients into matrix rows
Each equation $\rightarrow$ one row.

Example 1 — Fibonacci (Order 2 Recurrence)

Recurrence

$$F(n) = F(n-1) + F(n-2)$$

Step 1: State

$$S(n) = \begin{bmatrix} F(n) \\ F(n-1) \end{bmatrix}$$

Step 2: Next state

$$\begin{aligned} F(n+1) &= 1 \cdot F(n) + 1 \cdot F(n-1) \\ F(n) &= 1 \cdot F(n) + 0 \cdot F(n-1) \end{aligned}$$

Step 3: Transition matrix

$$M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

Example 2 — Simple Custom Recurrence

Recurrence

$$f(n) = 2f(n-1) + 3f(n-2)$$

Step 1: State

$$S(n) = \begin{bmatrix} f(n) \\ f(n-1) \\ f(n-2) \end{bmatrix}$$

Step 2: Next state

$$\begin{aligned} f(n+1) &= 1f(n) + 1f(n-1) + 1f(n-2) \\ f(n) &= 1f(n) + 0 + 0 \\ f(n-1) &= 0 + 1f(n-1) + 0 \end{aligned}$$

Step 3: Matrix

$$M = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Pattern You Should Burn Into Your Brain

For any k -order recurrence:

$$f(n) = c_1 f(n-1) + c_2 f(n-2) + \dots + c_k f(n-k)$$

Transition matrix form:

$$M = \begin{bmatrix} c_1 & c_2 & \dots & c_k \\ 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \end{bmatrix}$$

This is not a trick — it's state shifting.

Why This Matrix Works

Matrix multiplication is nothing but:

Linear combinations of current state values

And a linear recurrence is:

A fixed linear combination of previous values

So they match perfectly.

This is why only linear recurrences work.

Repeated Application = Matrix Power

Now observe:

$$S(n+1) = M \cdot S(n)$$

$$S(n+2) = M \cdot S(n+1) = M^2 \cdot S(n)$$

By induction:

$$S(n) = M^{n-1} \cdot S(1)$$

This is the entire reason matrix exponentiation exists.

```
// Multiply two 2x2 matrices
Matrix multiply(Matrix A, Matrix B) {
    Matrix C;
    memset(C.a, 0, sizeof(C.a));

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 2; k++) {
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
            }
        }
    }
    return C;
}
```

```
// Fast exponentiation of matrix
Matrix matrix_power(Matrix base, long long exp) {
    // Identity matrix
    Matrix result;
    result.a[0][0] = 1; result.a[0][1] = 0;
    result.a[1][0] = 0; result.a[1][1] = 1;

    while (exp > 0) {
        if (exp & 1)
            result = multiply(result, base);
        base = multiply(base, base);
        exp >>= 1;
    }
    return result;
}
```